

```
/*
```

```
COP3502C | Fall 2025 | Section 0004
```

```
Name: Austin Vodrazka
```

```
UCF ID: 5524103
```

```
*/
```

I wrote the code and tested it, then copied it to the doc. When I ran it, it printed:

```
toCString -> "abc"
```

```
1 2 -1 3 4 -1
```

```
1 2 3 4 5
```

```
2 99 1 2 99
```

1) 1) An alternate method of storing a string is to store each letter of the string in a single node of a

linked list, with the first node of the list storing the first letter of the string. Using this method of storage, no null character is needed since the next field of the node storing the last letter of the string would simply be a null pointer. Write a function that takes in a pointer to a linked list storing

a string and returns a pointer to a traditional C string storing the same contents. Make sure to dynamically allocate your string in the function and null terminate it before returning the pointer to the string. Assume that a function, length, exists already that you can call in your solution, that

takes in a pointer to a node and returns the length of the list it points to. The prototype for this function is provided below after the struct definition.

```
typedef struct node {  
    char letter;  
    struct node* next;  
} node;  
int length(node* head);  
char* toCString(node * head);
```

```
typedef struct node {  
    char letter;  
    struct node* next;  
} node;
```

```
int length(node* head); // given
```

```
static void fillChars(node* cur, char* s, int i){  
    if (cur == NULL){ s[i] = '\0'; return; }  
    s[i] = cur->letter;  
    fillChars(cur->next, s, i + 1);  
}
```

```

char* toCString(node* head){
    int n = length(head);
    char* s = (char*)malloc((n + 1) * sizeof(char));
    fillChars(head, s, 0);
    return s;
}

```

2) Suppose we have a linked list implemented with the structure below. Write a function that will take in a pointer to the head of a list and inserts a node storing -1 after each even value in the list.

If the list is empty or there are no even values in the list, no modifications should be made to the list. (For example, if the initial list had 2, 6, 7, 1, 3, and 8, the resulting list would have 2, -1, 6, -1, 7, 1, 8, -1.)

```

typedef struct node {
    int data;
    struct node* next;
} node;
void markEven(node *head);

```

```

typedef struct node {
    int data;
    struct node* next;
} node;

```

```

void markEven(node* head){
    if (head == NULL) return;
    if (head->data % 2 == 0){
        node* t = (node*)malloc(sizeof(node));
        t->data = -1;
        t->next = head->next;
        head->next = t;
        markEven(t->next); // skip the new node
    } else {
        markEven(head->next);
    }
}

```

3) Suppose we have a linked list implemented with the structure below. The function below takes

in a pointer, head, to a linked list which is guaranteed to store data in strictly ascending order. If the list doesn't contain the value 3, the function should create a struct node storing 3 in its data component, insert the node so that the list pointed to by head stores its data, including 3, in

strictly ascending order, and returns a pointer to the front of the resulting list. If a node already exists storing 3 in the list pointed to by head, then return head and make no changes to the list.

```
typedef struct node {
    int data;
    struct node* next;
} node;

node* addValue3(node* head){
    if (head == NULL || head->data > 3){
        node* tmp = (node*)malloc(sizeof(node));
        tmp->data = 3;
        tmp->next = head;
        return tmp;
    }
    if (head->data == 3) return head;

    node* cur = head;
    while (cur->next != NULL && cur->next->data < 3)
        cur = cur->next;

    if (cur->next != NULL && cur->next->data == 3) return head;

    node* tmp = (node*)malloc(sizeof(node));
    tmp->data = 3;
    tmp->next = cur->next;
    cur->next = tmp;
    return head;
}
```

4) Given the linked list structure named node, defined in lines 1 through 4, and the function named

eFunction defined in lines 6 through 14, answer the questions below.

```
1 typedef struct node {
2 int data;
3 struct node * next;
4 } node;
5
6 node* eFunction(node* aNode){
7 if(aNode == NULL) return NULL;
8 if(aNode->next == NULL) return aNode;
9
10 node* rest = eFunction(aNode->next);
11 aNode->next->next = aNode;
```

```
12 aNode->next = NULL;
13 return rest;
14 }
```

eFunction (concept questions)

(a) Yes, it's recursive.

(b) It reverses the linked list and returns the new head.

(c) Setting aNode->next = NULL breaks the old forward link, so the new tail doesn't form a cycle.

5) The structure of each node of a singly linked list is shown below.

```
typedef struct node {
int data;
struct node* next;
} node;
```

Write a function insertAfterN, that takes the head of a linked list, and two integers M and N (M ≠ N) and inserts M after all the nodes containing N.

For example, if M = 200 and N = 6, the linked list 3, 6, 4, 6, 6, 5 will be changed to 3, 6, 200, 4, 6, 200, 6, 200, 5.

```
void insertAfterN(node* head, int M, int N);
```

```
typedef struct node {
    int data;
    struct node* next;
} node;
```

```
void insertAfterN(node* head, int M, int N){
    if (head == NULL) return;
    if (head->data == N){
        node* t = (node*)malloc(sizeof(node));
        t->data = M;
        t->next = head->next;
        head->next = t;
        insertAfterN(t->next, M, N); // continue after the inserted node
    } else {
        insertAfterN(head->next, M, N);
    }
}
```